

A large, blossoming pink cherry tree stands on a grassy bank next to a calm body of water. The tree's branches are heavily laden with bright pink flowers. The sky is a mix of soft orange, yellow, and blue, suggesting a sunset or sunrise. The water reflects the colors of the sky and the pink blossoms. In the background, there are some blue and white flowers on the bank.

Implementação de Árvores B em Memória Secundária

Charlie Tanfoll - 16827968

Marina Cintra Queiroz - 17074404

Arquitetura e Estrutura de Dados

1 Classes

- Node: um nó = um bloco em disco
- Btree: mSearch, insertB, deleteB + acesso ao disco
- Header: raiz, ordem e lixeira
- caminhoDaBusca: retorno do mSearch
- ArquivoLivros: registros de livros (tamanho fixo)

2 Estrutura do nó

- Vetor contíguo de $2m$ inteiros
- Formato: $n \mid K1...K(m-1) \mid A0...A(m-1)$
- Filho -1 = folha
- Posição: $\text{header} + \text{rrn} \times 2m \times 4$ bytes
- Uma leitura/escrita por nó

3 Dropout

- Encapsulamento: header e arquivo private
- Sem variáveis globais
- Raiz e ordem gravadas no início do .idx
- Ordem fora dos nós
- Arquivo aberto durante toda a execução

Gerenciamento de I/O em Disco

● Estratégia de leitura e escrita:

- Arquivo .idx aberto uma vez no construtor e fechado no destrutor
- Sem open/close por nó
- Acesso direto por offset: seekg / seekp
- Nó rrn fica em $\text{sizeof}(\text{header}) + \text{rrn} \times 2m \times 4$
- flush() após cada gravação: o disco fica sempre consistente

● Transferência em bloco

- O nó é um vetor contíguo de $2m$ inteiros
- 1 read ou 1 write por nó, sem ler chave por chave

● Memória principal x secundária

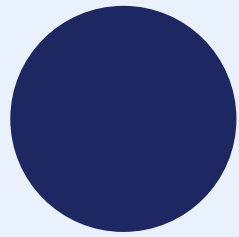
- Disco: header (raiz, ordem, lixeira) e todos os nós
- RAM: cópia do header, o nó em uso e a pilha do mSearch (raiz até a folha)
- O header em RAM é regravado no disco a cada mudança (nova raiz, lixeira) e no destrutor
- Cada nível descido = 1 leitura de disco
- Custo da busca: no máximo a altura da árvore

Gerenciamento de I/O em Disco

```
// leitura
file->seekg(posicao);
file->read((char*)buffer.data(),
          buffer.size() * sizeof(int));

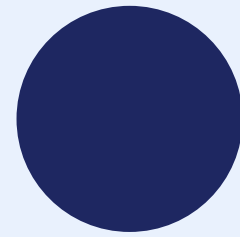
// escrita
file.seekp(posicao);
file.write((char*)no->buffer.data(),
           no->buffer.size() * sizeof(int));
```

Msearch



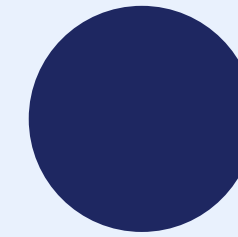
Abordagem iterativa

- Um único while (`rrnAtual != -1`), sem recursão
- Começa em `header.root`
- Termina ao achar a chave ou ao chegar num filho -1 (folha)



Retorno encapsulado

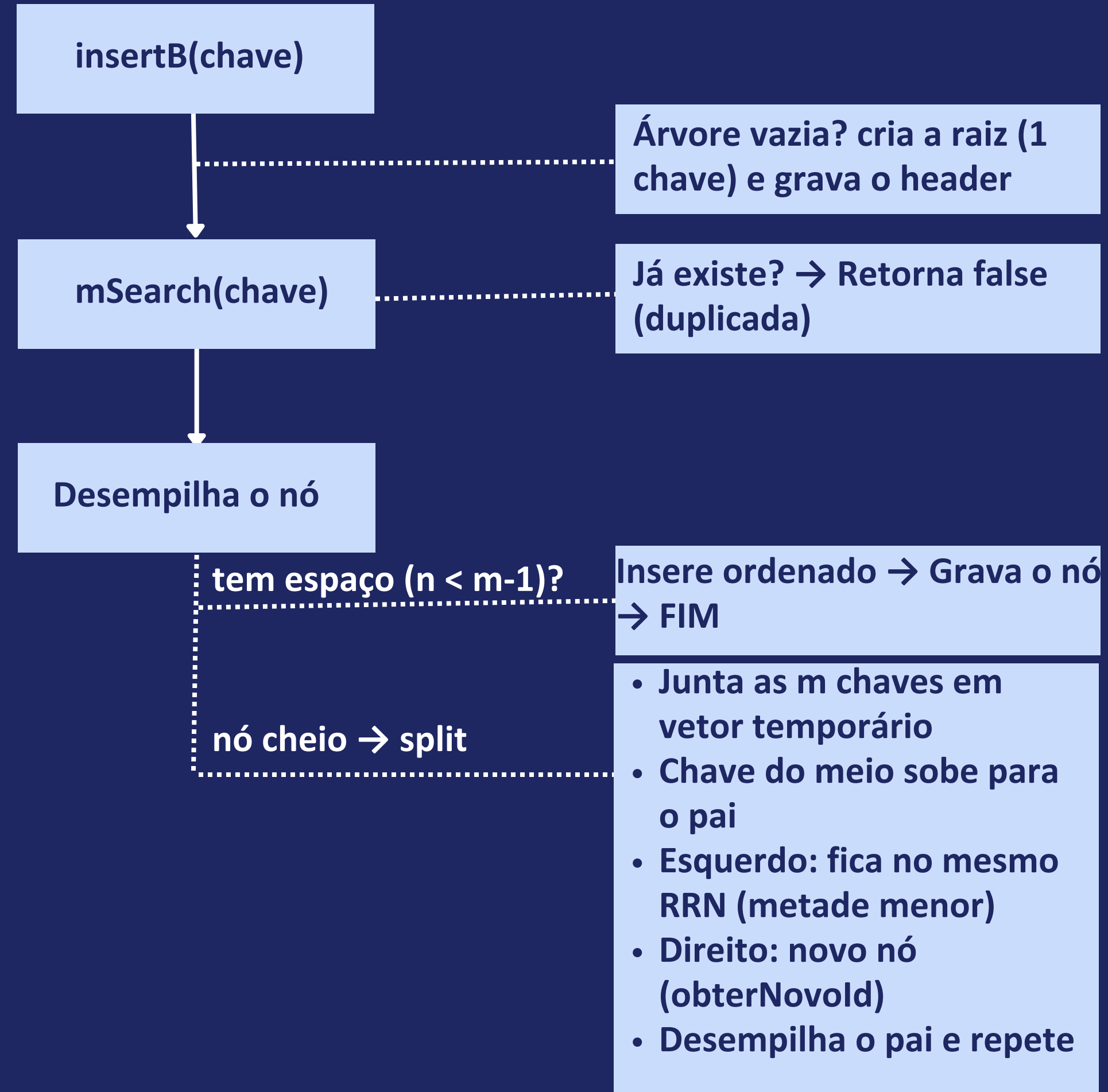
- Devolve a struct `caminhoDaBusca`:
- `encontrado`: true ou false
- `posicoesInternas`: índice usado em cada nó
- `nodesAcessados`: pilha dos nós visitados
- Sem cout dentro do método



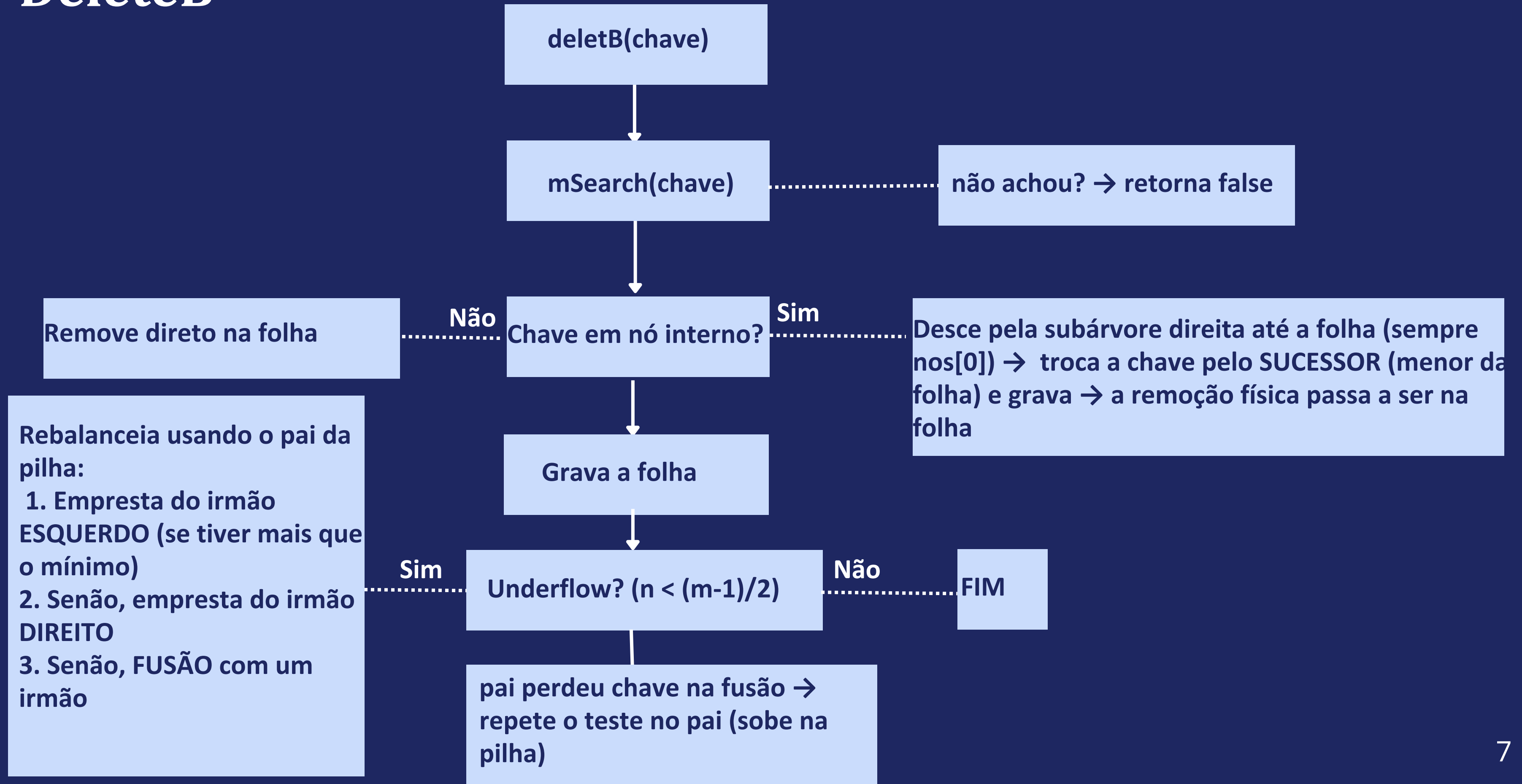
Pilha de caminho

- Cada nó lido é empilhado em RAM (`vector<unique_ptr<Node>>`)
- Pilha = caminho da raiz até onde a busca parou
- Se não achou: o topo é a folha e a posição é onde a chave entraria
- Custo: no máximo a altura da árvore em nós na RAM

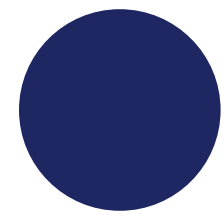
InsertB



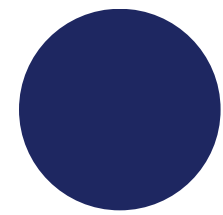
DeleteB



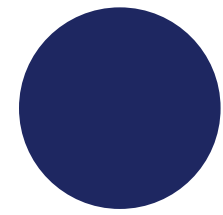
Desafios



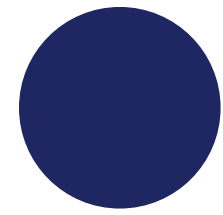
Ponteiros de arquivo em C++: `clear()` do estado de EOF antes de `seekg`, `seekp` vs `seekg`



Fusão e empréstimo: manter chaves e filhos alinhados ao deslocar vetores



Chave do sucessor: estender a pilha do `mSearch` até a folha



Índice sem ponteiro para o registro: busca sequencial no arquivo principal

Demonstração